



CO3-AT1-Debugging and Optimization

[← Back](#)

QUESTIONS

Q1ATM Withdrawal – Exception
Handling Debugging
10 marks**Q2**Hospital Registration –
NullPointerException
Debugging
10 marks**Q3**E-Commerce Product Search –
Array Exception Debugging
10 marks**Q4**Railway Reservation –
Synchronization Debugging
10 marks**Q5**Bank Account – Race Condition
Debugging

Q4 Railway Reservation – Synchronization Debugging

10 marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {  
  
    int seats = 1;  
  
    void bookSeat(String customer) {  
  
        if (seats > 0) {
```

10 marks

5/5 answered

```
System.out.println(customer
+ " is booking a seat");

try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}

seats--;

System.out.println(customer
+ " booking successful");

} else {
    System.out.println(customer
+ " - No seat available");
}
}

class Customer extends Thread {

    Reservation reservation;
    String customerName;

    Customer(Reservation reservation, String customerName) {
        this.reservation = reservation;
        this.customerName = customerName;
    }
}
```

```
public void run() {  
    reservation.bookSeat(customerName);  
}  
}  
  
class RailwayReservation {  
  
    public static void main(String[] args) {  
  
        Reservation reservation = new Reservation();  
  
        Customer c1 =  
            new Customer(reservation, "Customer 1");  
  
        Customer c2 =  
            new Customer(reservation, "Customer 2");  
  
        c1.start();  
        c2.start();  
    }  
}
```

The corrected program must protect the shared seat resource using synchronization.

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class RailwayReservation  
2 {  
3     public static void main(String[] args)
```

Test Input

```

4      {
5          Reservation r=new Reservation();
6          Customer c1=new Customer(r,"Customer 1");
7          Customer c2=new Customer(r,"Customer 2");
8          c1.start();
9          c2.start();
10     }
11 }
12 class Reservation{
13     int seats =1;
14     synchronized void bookSeat(String customer){
15         if(seats>0){
16             System.out.println(customer+" is booking
17             try{
18                 Thread.sleep(100);
19             }catch(Exception e){
20                 System.out.println("Thread interrup
21             }
22             seats--;
23             System.out.println(customer+" booking
24         }else{
25             System.out.println(customer+" -No seat
26         }
27     }
28 }
29 class Customer extends Thread{
30     Reservation reservation;
31     String name;
32     Customer(Reservation r,String name){
33         reservation = r;
34         this.name = name;
35     }
36     public void run(){

```

stdin input (optional)

Output

Customer 1 is booking
a seat
Customer 1 booking
successful
Customer 2 -No seat
available

⌚ Pending manual review

```
37         reservation.bookSeat(name);  
38     }  
39 }
```

✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.